Taxi Tap

Digital Wallet for Minibus Taxi Fares

To: Group 4 team; copy to Ms. Amenda Raganya (Project Supervisor)

From: Lethukuthula Gumede (Group Lead)

Module: Software Engineering 2 (SPM622) – Week 3

Date: 15 October 2026

Live doc: Tech Stack & Project Structure (Claude Docs)

Project plan: Linear: https://linear.app/taxi-tap (team TAX)

TECH STACK AND PROJECT STRUCTURE

Taxi Tap: Digital Wallet for Minibus Taxi Fares

Team and Areas

Member	Area
Lethukuthula Gumede (Group Lead)	GitHub, project management, builds, setup scripts, UI design, Google Maps
Christoffel Du Toit	QR code, front-end screens, ERD and schema, testing, compiling submissions
Londa Mavimbela	NFC and server actions (wallets, ledger, payments, fares)

Contents

#	Section
1	Summary of the stack
2	Architecture
3	Repository and folder structure
4	Data model (input for the ERD)
5	Roles for Week 3 and Week 4
6	Workflow: from a Linear issue to merged code
7	Local setup
8	Decisions, limits and open items

1. SUMMARY OF THE STACK

Taxi Tap is one Next.js web app (installable on phones) on a Neon Postgres database, accessed through Drizzle ORM, with BetterAuth for sign-in and Linear for planning. There is no Docker: each developer gets their own Neon branch, and `npm run setup` makes every laptop identical. All money is simulated.

Layer	Tool	Why we chose it
Front end	Next.js (React, TypeScript), App Router	One codebase for the customer, reader, merchant and owner screens; runs in the phone browser over HTTPS
UI components	shadcn/ui (Tailwind CSS)	Accessible components we own in the repo; quick to theme in the Taxi Tap purple style
Server code	Next.js server actions and route handlers	No separate API server; each action checks the user and role first
Database	Neon (serverless Postgres), one branch per developer	Real SQL with foreign keys and transactions; free tier; branches replace Docker
ORM and migrations	Drizzle ORM + drizzle-kit	Type-safe tables in TypeScript; generated SQL migration files are our schema script
Authentication	BetterAuth with the Drizzle adapter	Email and password sign-in, sessions and roles, stored in our Neon database
Maps	Google Maps JavaScript API	Shows routes and the GPS point of each reader payment
QR	Camera QR scanner library + QR generator (to be picked by Chris)	Main payment path; works on Android and iOS
NFC	Web NFC (`NDEFReader`)	Optional tag read on Android Chrome only; simulate-tap button elsewhere
Hosting	Vercel (app) + Neon (database)	Free tiers, HTTPS by default, preview link per pull request
Planning	Linear (team TAX) linked to GitHub	Issues, branches and pull requests linked automatically
Code	GitHub (one repo)	Version control evidence for rubric criterion 6

Table 1: Taxi Tap technology stack

Decision (15 Oct): Convex and Docker Desktop are dropped. We use Neon for the database and Drizzle to talk to it; the two work together, so we did not have to choose one.

2. ARCHITECTURE

The phone loads the Next.js screens from Vercel. Screens call server actions in the same app, and those actions use Drizzle to read and write Neon. BetterAuth runs inside the app too and stores its users and sessions in the same database, so there is no separate API server to build.

Screens call server actions; money moves only through transfer()

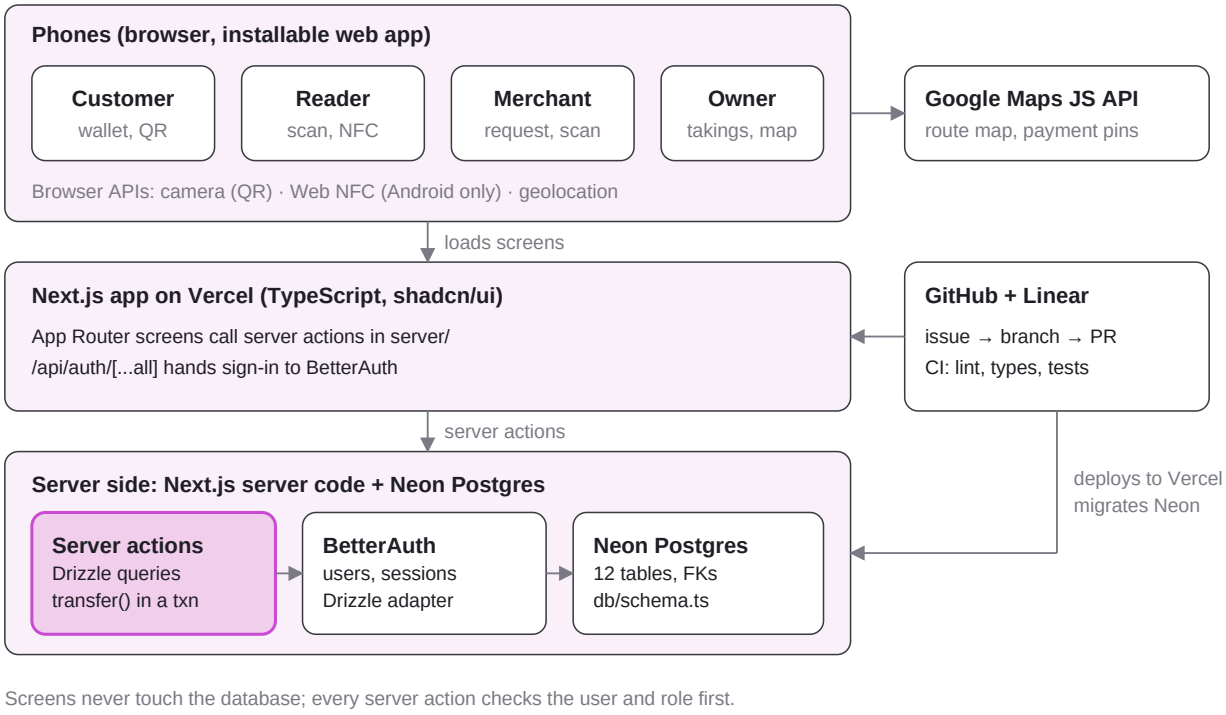


Figure 1: Taxi Tap architecture (3 layers, 2 outside services)

A payment is one call: the reader screen sends the QR token (or NFC tag) to a server action. The action checks the token, works out the fare and runs `transfer()` inside one `db.transaction`. The reader gets success or failure back in the same call. We use Drizzle's `neon-serverless` driver because the HTTP driver cannot run these transactions.

3. REPOSITORY AND FOLDER STRUCTURE

One GitHub repo, `taxi-tap`. The Next.js app sits at the root, the database code in `db/` and `drizzle/`, the server actions in `server/`, the setup scripts in `scripts/`, and the capstone evidence folders in `capstone/`. Each folder has one owner who reviews every pull request that touches it.

```
taxi-tap/
├─ app/                          Next.js App Router (screens)
│  ├─ (auth)/sign-in, sign-up    Login and register
│  ├─ (customer)/wallet, pay, history
│  ├─ (reader)/trip, scan        Boarding reader (driver phone)
│  ├─ (merchant)/request, scan   Traders and service providers
│  ├─ (owner)/dashboard, map     Takings per taxi and driver, route map
│  └─ api/auth/[...all]/route.ts BetterAuth handler
├─ components/
│  ├─ ui/                        shadcn/ui components (themed)
│  ├─ qr/                        QrCode, QrScanner
│  ├─ nfc/                       NfcReader, SimulateTap
│  └─ maps/                      RouteMap, PaymentPins
├─ db/
│  ├─ schema.ts                  Our tables (from the ERD)
│  ├─ auth-schema.ts             BetterAuth tables (generated)
│  └─ index.ts                   The one Drizzle client
├─ drizzle/                      SQL migrations (generated, committed)
├─ server/                       Server actions ("use server")
│  ├─ auth.ts                    BetterAuth config and requireRole()
│  ├─ organisations.ts           Associations, owners, drivers, vehicles, stalls
│  ├─ wallets.ts, ledger.ts      Balances and transfer()
│  ├─ qr.ts                      Issue and check signed QR tokens
│  ├─ payments.ts                Reader and merchant payments
│  └─ routes.ts, fares.ts        Routes, fare bands, fare engine
├─ lib/                          Pure TypeScript helpers (unit tested)
├─ tests/                        Vitest unit tests
├─ scripts/                      setup.mjs, doctor.mjs, seed.ts
├─ capstone/00_Project_Administration … 12_Assessment
├─ .github/workflows/ci.yml      Lint, type-check, tests on every PR
├─ drizzle.config.ts, .nvmrc     Migration settings, Node 22
└─ README.md, .env.example
```

Figure 2: Repository folder structure

Folder	Owner	Reviewer
`app/(auth)`, `app/(customer)`, `app/(merchant)`	Christoffel	Lethukuthula
`app/(reader)`, `components/qr`, `server/qr.ts`	Christoffel	Londa
`components/nfc`, `lib/nfc.ts`	Londa	Christoffel
`app/(owner)`, `components/maps`	Lethukuthula	Christoffel
`components/ui`, theme, design tokens	Lethukuthula	Christoffel
`db/schema.ts`, `drizzle/`	Christoffel (from the ERD)	Londa
`server/` actions (wallets, ledger, payments, fares, organisations), `scripts/seed.ts`	Londa	Lethukuthula
`server/auth.ts`, `scripts/setup.mjs`, `scripts/doctor.mjs`, `.github/`, `capstone/`	Lethukuthula	Londa

Table 2: Folder owners and reviewers

Boundary rules

Every pull request must follow these rules:

1. Screens never touch the database. They call server actions only, and only `server/` imports `db/`.
2. Every server action checks the signed-in user and their role first (`requireRole()`).
3. Money moves only through `transfer()`, which debits, credits and records the transaction inside one `db.transaction`.
4. Amounts are stored as whole cents (integers), never decimals.
5. A QR token is signed, carries its wallet ID and expires after 60 seconds; the server checks it, never the phone.
6. Each payment carries an idempotency key; a unique index on it stops double charges.
7. The database only changes through `db/schema.ts` and a generated migration in `drizzle/`. Never edit tables by hand in the Neon console.
8. No API keys or secrets in the repo. They live in Vercel environment settings and `.env.local`.

4. DATA MODEL (INPUT FOR THE ERD)

The ERD has 12 Postgres tables, written as Drizzle `pgTable` definitions in `db/schema.ts`. Every table has a `uuid` primary key and real foreign keys, so the database itself rejects broken links. BetterAuth's own tables (`user`, `session`, `account`, `verification`) are generated into `db/auth-schema.ts`; our `profiles` table links to `user.id`. A draft schema is in the starter pack (it type-checks and generates a migration). Chris owns the final version, and the generated SQL file in `drizzle/` is the schema script for Week 3.

Table	Key fields	Links to	Indexes
profiles	authUserId, name, phone, role (customer, driver, owner, association_admin, merchant_staff)	BetterAuth user	by_authUserId
organisations	name, type (association, business), parentId	organisations (a business points to its association)	by_parent
memberships	profileId, organisationId, role	profiles, organisations	by_profile, by_organisation
vehicles	registration, ownerOrgId, routeId	organisations, routes	by_owner
stalls	name, ownerOrgId	organisations	by_owner
routes	associationId, name, fareModel (flat, ladder)	organisations	by_association
fareBands	routeId, label (e.g. "to Berea"), amountCents, sortOrder	routes	by_route
trips	vehicleId, driverId, routeId, startedAt, endedAt	vehicles, profiles, routes	by_vehicle, by_driver
wallets	ownerType (profile, organisation), ownerId, balanceCents	profiles or organisations	by_owner

Table	Key fields	Links to	Indexes
transactions	fromWalletId, toWalletId, amountCents, kind (topup, fare, merchant), method (qr, nfc, simulated), quantity, tripId, fareBandId, staffId, lat, lng, idempotencyKey	wallets, trips, fareBands, profiles	by_from, by_to, by_trip, by_idempotencyKey
qrTokens	walletId, nonce, expiresAt, usedAt	wallets	by_nonce
nfcTags	tagSerial, walletId, active	wallets	by_tagSerial

Table 3: Draft tables for the ERD

The relationships follow the organisation structure we agreed:

- One association has many owner businesses. Each business has many drivers or staff, vehicles and stalls.
- One route has many fare bands. One trip runs on one route.
- Each fare is recorded as one transaction. It debits the customer's wallet and credits the owner business's wallet, and is tagged with the trip, driver and fare band.
- QR tokens and NFC tags both resolve to a wallet. The payment path after that is the same.

5. ROLES FOR WEEK 3 AND WEEK 4

Week 3 closes tomorrow, Friday 16 Oct at 12pm. Week 4 runs 17 to 23 Oct. Every item below is a Linear issue in team TAX, assigned to its owner.

Member	Area	Week 3 (due 16 Oct)	Week 4 (17–23 Oct)
Lethukuthula	GitHub, project management, builds, UI design, Google Maps	Skeleton and repo TAX-26, setup scripts and Neon branches TAX-93, Linear–GitHub link TAX-92, BetterAuth TAX-31, role helpers TAX-33, deploy TAX-27, CI TAX-28, design system TAX-85, wireframes TAX-86, Ch3 TAX-70, Ch4 4.2–4.5 TAX-72	GPS stamp TAX-53, Google Maps TAX-90, usability test TAX-65, Ch4 4.6–4.8 TAX-73, video TAX-80, release tags TAX-30
Christoffel	QR code, front-end screens, ERD, testing, compiling submissions	ERD and schema TAX-89, login screens TAX-32, top-up TAX-37, history TAX-39, test plan TAX-60, test cases TAX-61, SRS TAX-71, compile Week 3 PDF TAX-84	QR tokens TAX-41, QR screen TAX-42, scanner TAX-43, reader screens TAX-48 TAX-50 TAX-52, merchant TAX-55, test cycle 2 TAX-64, Ch5 TAX-74, demo script TAX-81
Londa	NFC and server actions	Organisations TAX-34, vehicles TAX-35, wallets TAX-36, ledger TAX-38, ledger tests TAX-40, routes and fares TAX-47	NFC read TAX-46, NFC tags TAX-91, payment API TAX-44, fare engine TAX-49, fare tests TAX-51, merchant payments TAX-56, seed TAX-29, reliability test TAX-66

Table 4: Week 3 and Week 4 work per member

TAX-89 (ERD and schema) blocks Londa's server actions, so Chris should merge it first. TAX-93 (setup scripts and Neon branches) unblocks everyone's local setup, so Lethukuthula merges it today.

Londa's Week 3 issues are due tomorrow. If she can't finish them, Lethukuthula picks up the ledger (TAX-38) first, because every payment depends on it.

6. WORKFLOW: FROM A LINEAR ISSUE TO MERGED CODE

Every piece of code starts as a Linear issue and ends as a reviewed pull request that closes it. This gives us the commit, branch and review evidence that rubric criterion 6 marks.

1. Pick your next issue in Linear (team TAX) and move it to In Progress.
2. Copy the issue's branch name in Linear (Ctrl/Cmd + Shift + .) and create the branch from `main`, e.g. `tax-41-signed-short-lived-qr-token`.
3. Commit small and often. Each message says what changed and names the issue, e.g. `Add 60-second expiry to QR tokens (TAX-41)`. Never just "update".
4. Open a pull request into `main`. Put `Fixes TAX-41` in the description so Linear links the PR and moves the issue to Done when it merges.
5. The folder owner's reviewer (see Table 2) approves it. CI must pass: lint, type-check and Vitest.

6. Squash-merge, delete the branch, and check that the Vercel preview still works on a phone.

Rules for `main`: protected, no direct pushes, at least one approval. Release tags are `v0.1` on 16 Oct, `v0.2` on 23 Oct and `v1.0` on 28 Oct (TAX-30).

Each of us commits in every sprint, including documentation-only weeks. Chapters and evidence go in `capstone/` as their own pull requests.

7. LOCAL SETUP

No Docker. Each of us gets our own Neon database branch, and one script installs, checks and prepares everything, so the app runs the same on every laptop. Setup takes about 10 minutes.

1. Install Node.js 22 LTS, Git and VS Code. `node -v` must start with `v22` (the repo's `.nvmrc` says 22). Accept the GitHub invite to `taxi-tap`.
2. Clone the repo and `cd taxi-tap`.
3. In the Neon console, open the Taxi Tap project, go to Branches, create `dev-<yourname>` from `main` and copy its pooled connection string.
4. Run `npm run setup`. The first run creates `.env.local` and stops.
5. Fill in `.env.local` (your Neon string; Lethukuthula shares the other secrets), then run `npm run setup` again. It installs exact package versions with `npm ci`, tests the database connection, applies migrations and loads demo data.
6. Run `npm run dev` and open `http://localhost:3000`.
7. To test the camera or NFC on a phone you need HTTPS. Use the Vercel preview link of your pull request, or `next dev --experimental-https` on the same Wi-Fi.

Something broken? `npm run doctor` checks Node, the lockfile, packages, `.env.local` and the database, changes nothing, and says what to fix. Before every pull request run `npm run check` (lint, type-check, tests), which is exactly what CI runs.

Command	What it does
`npm run setup`	Install, check, migrate and seed (safe to re-run)
`npm run doctor`	Diagnose without changing anything
`npm run check`	Lint + type-check + tests, same as CI
`npm run db:generate`	After editing `db/schema.ts`, write a new SQL migration to `drizzle/`
`npm run db:migrate`	Apply migrations to your Neon branch
`npm run db:studio`	Browse your data in the browser
`npm run db:seed`	Reload demo data

Table 5: Daily commands

Variable	Where it lives	Owner
`DATABASE_URL`	`.env.local` (your own Neon branch), Vercel (the `main` branch)	each developer / Lethukuthula
`BETTER_AUTH_SECRET`, `BETTER_AUTH_URL`	`.env.local`, Vercel	Lethukuthula

Variable	Where it lives	Owner
`QR_SIGNING_SECRET`	`.env.local`, Vercel	Christoffel
`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`	`.env.local`, Vercel (key restricted to our domains); optional locally	Lethukuthula

Table 6: Environment variables

`.env.example` lists every variable with no values. Real values never go into Git. If your Neon branch gets messy, reset it from `main` in the Neon console and run `npm run setup -- --skip-install`.

8. DECISIONS, LIMITS AND OPEN ITEMS

Item	Status	Note
Neon + Drizzle; Convex dropped	Decided 15 Oct	We need SQL for the ERD and schema script; Convex is not SQL
No Docker; Neon branch per developer + setup/doctor scripts	Decided 15 Oct	Same Node version, lockfile and checks on every laptop
Drizzle `neon-serverless` driver (WebSockets)	Decided 15 Oct	The ledger needs `db.transaction`; the HTTP driver cannot do it
Hosting on Vercel + Neon	Proposed	Free tiers and HTTPS; confirm or swap at stand-up
QR library choice	Open (Chris)	Must scan on Android Chrome and iOS Safari; pick by 17 Oct
Google Maps billing	Open (Lethukuthula)	Google Cloud needs a billing account; restrict the key and watch usage
Web NFC	Limit	Android Chrome over HTTPS only; iOS users get the simulate-tap button
Simulated funds only	Limit (topic acceptance)	No bank or payment provider; a licensed partner would hold real money
Ch4 4.5 "justified stack"	Uses this doc	Lethukuthula turns the Summary table into the chapter text for Week 3

Table 7: Decisions, limits and open items

Sources

Drizzle – Connect Neon. Available at: https://orm.drizzle.team/docs/connect-neon (neon-http vs neon-serverless, transactions).

Better Auth – Drizzle adapter. Available at: https://www.better-auth.com/docs/adapters/drizzle (adapter setup and schema generation).

Neon – Branching. Available at: https://neon.com/docs/introduction/branching (one database branch per developer).